

UNIVERSITÉ DE LORRAINE

Rapport de Preuve et Dédution Automatique

Implémentation de la méthode des connexions en Prolog

Étudiants :

Mathéo VIGNÈRES

Guillaume POTEL

Enseignant :

M. Didier GALMICHE

Année universitaire 2025-2026

Table des matières

1	Le socle commun : Logique Propositionnelle et Premier Ordre	2
1.1	Définition des règles (Alpha et Bêta)	2
1.1.1	Le prédicat principal : <code>etiqueter/7</code>	2
1.1.2	Les règles de type Alpha (α)	2
1.1.3	Les règles de type Bêta (β)	2
1.1.4	Cas de la négation et des feuilles	3
1.2	Structures de données : Représentation des arbres	3
1.2.1	Une structure récursive uniforme	3
1.2.2	Polyvalence de la structure	4
1.3	Prédicats utilitaires	4
1.3.1	Identification des littéraux et opérateurs	4
1.3.2	Substitution de termes	4
1.3.3	Gestion de l’affichage et de la trace	5
1.4	Le module principal et l’exécution (<code>main.pl</code>)	5
2	En Logique Propositionnelle	5
2.1	Construction de l’arbre syntaxique indexé	6
2.1.1	Structure de l’étiquette enrichie	6
2.1.2	Initialisation et Génération	6
2.1.3	Récupération par les accesseurs	6
2.2	Génération de l’arbre des chemins	6
2.2.1	Le développement de chemin	7
2.2.2	Traitement des règles α et β	7
2.2.3	Sémantique de l’Arbre des Chemins	7
2.3	Recherche de connexions et validation	7
2.3.1	Définition d’une connexion	7
2.3.2	Validation globale des chemins	8
2.3.3	Conclusion de la preuve	8
3	En Logique du Premier Ordre (LPO)	8
3.1	Adaptations structurelles pour la LPO	9
3.1.1	Traçabilité des variables : <code>var(Position, V)</code>	9
3.1.2	Transparence de l’arbre des chemins	9
3.2	La gestion de la Multiplicité	9
3.2.1	La boucle de recherche itérative	9
3.2.2	Génération des copies multiples	10
3.3	Unification et Graphe de Dépendance	10
3.3.1	L’Unification intelligente	10
3.3.2	Cohérence globale et Backtracking	11
3.3.3	Validation finale : Le Graphe de dépendance	11
	Exemples et résultats	11

1 Le socle commun : Logique Propositionnelle et Premier Ordre

Dans cette première partie, nous présentons les éléments de base qui sont partagés entre la logique propositionnelle et la logique du premier ordre.

1.1 Définition des règles (Alpha et Bêta)

La méthode des connexions s'appuie sur l'application de règles appelée *alpha* et *beta*. Nous avons implémenté ces règles via un prédicat central : `etiqueter/7`.

Ce prédicat a pour rôle de parcourir la formule initiale pour construire un arbre indexé, où chaque nœud contient le type de règle appliquée ($\alpha, \beta, \gamma, \delta$), sa polarité, et un index unique.

1.1.1 Le prédicat principal : `etiqueter/7`

La signature du prédicat est la suivante :

```
1 etiqueter(+Formule, +Polarite, +IndexIn, -IndexOut, +Multiplicite,  
+Suffixe, -Arbre).
```

- **Polarité** : 1 pour une formule affirmée, 0 pour une formule niée.
- **IndexIn / IndexOut** : Un mécanisme de compteur permettant d'attribuer un identifiant unique (ex : $a0, a1, \dots$) à chaque nœud de l'arbre.
- **Arbre** : La structure de sortie représentant l'arbre syntaxique indexé sous forme de nœud ou de feuille.

1.1.2 Les règles de type Alpha (α)

Les règles α correspondent aux décompositions de nature conjonctive (linéaire). Dans notre implémentation, une règle α produit deux sous-arbres qui devront tous deux être validés. Par exemple, pour la conjonction de polarité 1 ($A \wedge B, 1$) ou l'implication de polarité 0 ($A \rightarrow B, 0$) :

```
1 % Exemple : Implication niee (A -> B, 0)  
2 etiqueter(A impl B, 0, IndexIn, IndexOut, M, Suffix,  
3     noeud(etiq_formule(alpha, alpha1, alpha2, A impl B, 0,  
4         NomComplet),  
5         [ArbreA, ArbreB])) :-  
6     construire_nom(IndexIn, Suffix, NomComplet),  
7     Index1 is IndexIn + 1,  
8     etiqueter(A, 1, Index1, IndexMid, M, Suffix, ArbreA),  
    etiqueter(B, 0, IndexMid, IndexOut, M, Suffix, ArbreB).
```

1.1.3 Les règles de type Bêta (β)

Les règles β représentent des décompositions disjonctives (branchement). Elles créent une bifurcation dans l'arbre des chemins. Nous retrouvons ici la disjonction de polarité 1 ($A \vee B, 1$) ou l'implication de polarité 0 ($A \rightarrow B, 1$) :

```

1 % Exemple : Disjonction affirmée (A ou B, 1)
2 etiqueter(A ou B, 1, IndexIn, IndexOut, M, Suffix,
3     noeud(etiq_formule(beta, beta1, beta2, A ou B, 1,
4         NomComplet),
5         [ArbreA, ArbreB])) :-
6     construire_nom(IndexIn, Suffix, NomComplet),
7     Index1 is IndexIn + 1,
8     etiqueter(A, 1, Index1, IndexMid, M, Suffix, ArbreA),
9     etiqueter(B, 1, IndexMid, IndexOut, M, Suffix, ArbreB).

```

1.1.4 Cas de la négation et des feuilles

La négation est traitée de manière transparente en inversant la polarité du reste de la formule. Ce processus se poursuit jusqu'à atteindre un **littéral** (atome). À ce stade, le prédicat crée une feuille, marquant la fin de la branche syntaxique.

```

1 % Traitement des atomes (Feuilles)
2 etiqueter(A, Polarite, Index, IndexOut, _, Suffix,
3     feuille(etiq_formule(atome, none, none, A, Polarite,
4         NomComplet)))) :-
5     est_litteral(A),
6     construire_nom(Index, Suffix, NomComplet),
7     IndexOut is Index + 1.

```

1.2 Structures de données : Représentation des arbres

Pour représenter un arbre dans notre projet, nous avons choisi une structure récursive polyvalente capable de représenter aussi bien l'arbre syntaxique indexé que l'arbre des chemins.

1.2.1 Une structure récursive uniforme

Un arbre est défini de manière inductive par deux constructeurs principaux :

- `noeud(Etiquette, ListeFils)` : Représente un point de divergence ou de linéarité dans la formule. Contrairement à un arbre binaire classique, nous utilisons une liste de fils pour permettre une plus grande flexibilité (notamment pour la gestion de la multiplicité en LPO).
- `feuille(Etiquette)` : Représente un point terminal, (un littéral).

L'étiquette contient les métadonnées nécessaires au traitement : pour l'arbre syntaxique, il s'agit du prédicat `etiq_formule` vu précédemment. Pour l'arbre des chemins, elle contient l'ensemble des littéraux collectés.

```

1 % Verification du type de noeud
2 est_feuille(feuille(_)).
3 est_noeud(noeud(_, _)).
4
5 % Accesseurs pour l'extraction de donnees
6 noeud_etiquette(noeud(E, _), E).
7 noeud_fils(noeud(_, Fils), Fils).
8 feuille_etiquette(feuille(E), E).

```

1.2.2 Polyvalence de la structure

Cette implémentation est utilisée à deux étapes clés du programme :

1. **L'Arbre Indexé** : Chaque nœud correspond à une sous-formule et chaque feuille à un atome avec sa polarité. C'est la base de la décomposition logique.
2. **L'Arbre des Chemins** : Il s'agit d'une transformation de l'arbre précédent où chaque nœud représente l'état des chemins à un point donné de l'exploration.

Cette uniformité de structure facilite l'écriture des algorithmes de parcours de profondeur, car les mêmes prédicats de navigation peuvent être réutilisés sur les deux types d'arbres.

1.3 Prédicats utilitaires

Pour soutenir les étapes de décomposition et de recherche de preuves, nous avons développé un ensemble d'outils. Ces prédicats gèrent la syntaxe logique, la manipulation des variables et les mécanismes d'affichage.

1.3.1 Identification des littéraux et opérateurs

Un point crucial de la méthode des connexions est de savoir quand arrêter la décomposition d'une formule. Le prédicat `est_litteral/1` permet d'identifier si un terme est un atome (propositionnel ou prédicat du premier ordre) en vérifiant qu'il n'est pas un connecteur logique réservé.

```
1 % Un terme est un littéral s'il n'est pas un connecteur logique
2 est_litteral(A) :-
3     atom(A), !.
4 est_litteral(A) :-
5     compound(A),
6     functor(A, Foncteur, _),
7     \+ connecteur(Foncteur).
```

Nous avons également défini les priorités d'opérateurs pour permettre une écriture naturelle des formules (ex : `p et q impl r`).

1.3.2 Substitution de termes

Particulièrement utilisé en logique du premier ordre pour l'application des règles γ (universelles) et δ (existentielles), le prédicat `substituer/4` effectue un remplacement récursif d'un terme par un autre.

```
1 % Remplacement récursif de Ancien par Nouveau dans Terme
2 substituer(Ancien, Ancien, Nouveau, Nouveau) :- !.
3 substituer(Terme, Ancien, Nouveau, Resultat) :-
4     compound(Terme),
5     Terme =.. [F | Args],
6     maplist([Arg, ArgSubst]>>(substituer(Arg, Ancien, Nouveau,
7         ArgSubst)), Args, ArgsSubst),
8     Resultat =.. [F | ArgsSubst].
9 substituer(Terme, _, _, Terme).
```

Ce prédicat utilise l'opérateur "univ" ($=.$) pour décomposer n'importe quel prédicat en une liste de ses arguments, permettant une substitution générique quelle que soit l'arité du terme.

1.3.3 Gestion de l'affichage et de la trace

Nous avons créé tout un module d'affichage pour suivre les traces de la méthode des connexions. Pour ne pas encombrer la logique métier, ces outils sont regroupés et traitent les points suivants :

- **Écriture symbolique** : Les prédicats `ecrire_formule/1` et `ecrire_type/1` traduisent les termes internes (ex : `impl`, `alpha`) en symboles mathématiques ($\rightarrow$, α , $\forall$, $\exists$).
- **Nettoyage des variables** : `nettoyer_formule/2` transforme les structures internes complexes (utilisées pour l'unification) en noms lisibles pour l'utilisateur final.
- **Système d'Echo** : Nous avons réutilisé le système d'affichage donné pour le projet d'algorithme d'unification de Martelli-Montanari au 1er semestre, dans le cadre de l'UE *Logique et Modèles de Calculs*.

1.4 Le module principal et l'exécution (main.pl)

Pour orchestrer l'ensemble de ces outils et faciliter nos phases de tests, nous avons regroupé les points d'entrée du programme au sein d'un module central : `main.pl`.

Le prédicat principal du programme est `verifier(+Formule, +Logique)`, où l'argument `Logique` prend la valeur `prop` ou `lpo`. Il se charge d'exécuter séquentiellement les grandes étapes de la méthode des connexions :

1. La génération de l'arbre syntaxique indexé.
2. La construction de l'arbre des chemins.
3. La recherche de connexions et l'évaluation finale.

Pour nos tests et l'affichage, nous avons mis en place un mode "trace" via le prédicat `trace_verif`. Ce dernier active les variables d'affichage (système d'écho) avant de lancer l'évaluation, permettant de visualiser la construction des arbres, les étapes d'unification et le graphe de dépendance directement dans la console.

```

1 % Point d'entree avec affichage detaillé (trace)
2 trace_verif(Formule) :- trace_verif(Formule, prop).
3
4 trace_verif(Formule, Logique) :-
5     activer_echo,
6     verifier(Formule, Logique),
7     desactiver_echo.
```

2 En Logique Propositionnelle

Cette section détaille l'implémentation de la méthode des connexions restreinte à la logique propositionnelle.

2.1 Construction de l'arbre syntaxique indexé

La première étape de la méthode consiste à transformer la formule logique brute en un arbre syntaxique dont chaque nœud contient les informations nécessaires à la suite de la méthode. Cette structure, appelée **Arbre Indexé**, sert de base à toutes les étapes ultérieures (génération des chemins et recherche de connexions).

2.1.1 Structure de l'étiquette enrichie

Chaque nœud ou feuille de l'arbre contiennent une étiquette construite comme cela :

`etiq_formule(TypePrincipal, TypeSec1, TypeSec2, Formule, Polarité, Index)`

- **TypePrincipal** : Identifie la règle applicable (α, β pour la propositionnelle).
- **Types Secondaires** : Précisent le rôle des fils (ex : α_1 ou α_2).
- **Polarité** : Indique si la formule est affirmée (1) ou niée (0).
- **Index** : Un entier unique attribué selon un parcours en profondeur, permettant d'identifier précisément chaque position dans l'arbre.

2.1.2 Initialisation et Génération

Le prédicat `generer_arbre_indexe/2` lance le processus en initialisant la polarité à 0. Ce choix s'explique par la nature de la méthode des connexions : pour prouver qu'une formule est une tautologie, on cherche à montrer que sa négation (polarité 0) mène toujours à une contradiction.

```
1 % Initialisation de la construction
2 generer_arbre_indexe(Formule, Arbre) :-
3     regles_communes:etiqueter(Formule, 0, 0, _, none, none, Arbre).
```

Le prédicat fait appel au module `regles_communes` (décrit en Partie I) pour utiliser le prédicat d'étiquetage.

2.1.3 Récupération par les accesseurs

Pour manipuler cette structure complexe sans casser l'encapsulation, nous avons mis en place des accesseurs. Ils permettent d'extraire une information précise sans avoir à connaître l'ordre des arguments dans le terme `etiq_formule`.

```
1 % Exemples d'accesseurs
2 etiq_type_principal(etiq_formule(T, _, _, _, _, _), T).
3 etiq_formule(etiq_formule(_, _, _, F, _, _), F).
4 etiq_index(etiq_formule(_, _, _, _, _, I), I).
```

Nous avons mis en place ce système en prévoyant une potentielle évolution vers la logique des prédicats, sans avoir besoin de changer toute la logique métier.

2.2 Génération de l'arbre des chemins

Une fois l'arbre syntaxique indexé construit, l'étape suivante consiste à extraire l'ensemble des chemins sémantiques de la formule. Nous enregistrons ces chemins dans un arbre appelé **Arbre des chemins**.

2.2.1 Le développement de chemin

L'objectif est de parcourir l'arbre syntaxique et d'appliquer les règles de réduction de manière à isoler tous les ensembles de littéraux possibles qui, s'ils sont tous validés, prouvent la formule. Le cœur de cet algorithme réside dans le prédicat `developper/2`.

Ce prédicat manipule un *Ensemble* de nœuds issus de l'arbre indexé :

- **Cas de base** : Si l'ensemble ne contient plus que des feuilles (des littéraux), le chemin est complet. On crée une feuille `etiq_chemin_final` contenant la liste de ces littéraux.
- **Récursion** : On extrait le premier nœud de l'ensemble pour le traiter selon son type principal.

2.2.2 Traitement des règles α et β

La structure de l'arbre des chemins dépend directement de la nature du connecteur traité :

1. **Règle α (Conjonctive)** : Les composants de la formule α (ex : A et B pour $A \wedge B$) doivent appartenir au même chemin. On ajoute donc les fils à l'ensemble courant et on continue le développement linéairement.
2. **Règle β (Disjonctive)** : La formule β crée une bifurcation. Chaque fils de la règle β engendre un nouveau chemin indépendant.

2.2.3 Sémantique de l'Arbre des Chemins

Contrairement à l'arbre syntaxique qui représente la structure logique de la formule, l'arbre des chemins représente la **recherche de preuve** :

- Chaque nœud de cet arbre représente une étape de réduction (une application de règle).
- Chaque feuille de cet arbre représente un **chemin complet**.

Pour que la formule soit prouvée, chaque feuille de cet arbre des chemins devra contenir au moins une *connexion* (une paire de littéraux complémentaires). Cette structure permet une séparation claire entre la phase de décomposition logique et la phase de recherche de connexions que nous détaillerons dans la section suivante.

2.3 Recherche de connexions et validation

La dernière étape de la méthode des connexions en logique propositionnelle consiste à vérifier si l'arbre des chemins constitue une preuve valide. Pour qu'une formule soit prouvée, *tous* les chemins générés doivent être "fermés", c'est-à-dire qu'ils doivent contenir au moins une connexion.

2.3.1 Définition d'une connexion

Une connexion est une paire de littéraux (atomes) complémentaires présents sur un même chemin. Dans notre représentation, cela se traduit par deux feuilles ayant la même formule mais des polarités opposées (1 et 0).

Le prédicat `connexion/3` est chargé de trouver cette paire au sein d'une liste de feuilles. Il exploite le non-déterminisme de Prolog en utilisant `select/3` et `member/2` pour extraire deux éléments distincts, puis vérifie leurs propriétés via les accesseurs de l'étiquette :


```

1  % Recherche d'une paire complementaire
2  connexion(Feuilles, F1, F2) :-
3      % Extraction d'une premiere feuille
4      select(F1, Feuilles, Reste),
5      feuille_etiquette(F1, Etiquette1),
6      etiq_formule(Etiquette1, Symbole),
7      etiq_polarite(Etiquette1, Polarite1),
8
9      % Calcul de la polarite opposee attendue
10     Polarite2 is 1 - Polarite1,
11
12     % Recherche d'une seconde feuille correspondante
13     member(F2, Reste),
14     feuille_etiquette(F2, Etiquette2),
15     etiq_formule(Etiquette2, Symbole),
16     etiq_polarite(Etiquette2, Polarite2).

```

2.3.2 Validation globale des chemins

Pour valider la preuve entière, l'algorithme doit parcourir l'arbre des chemins jusqu'à ses feuilles. Ce parcours est réalisé par le prédicat `tous_chemins_connectes/1`.

Lorsqu'il rencontre un nœud de branchement, il délègue récursivement la vérification à tous ses fils grâce à `maplist/2`. Lorsqu'il atteint une feuille finale (`etiq_chemin_final`), il vérifie qu'une connexion existe pour ce chemin :

```

1  % Verification sur un chemin final
2  tous_chemins_connectes(feuille(etiq_chemin_final(Feuilles))) :-
3      chemin_connecte(Feuilles).
4
5  % Recursion sur l'arbre des chemins
6  tous_chemins_connectes(noeud(_, ListeFils)) :-
7      maplist(tous_chemins_connectes, ListeFils).

```

Une optimisation importante réside dans le prédicat `chemin_connecte/1` (qui appelle `connexion/3`). L'utilisation d'un *cut* (!) permet de stopper la recherche dès qu'une première connexion est trouvée sur le chemin courant. En effet, une seule connexion suffit à fermer un chemin, il est donc inutile d'explorer les autres paires possibles.

2.3.3 Conclusion de la preuve

Le point d'entrée de ce processus est `verifier_connexions/2`. Il unifie son résultat avec l'atome `valide` si le prédicat `tous_chemins_connectes` réussit sur l'arbre complet, confirmant ainsi que la matrice est fermée (et donc que la formule de départ est bien une tautologie). Dans le cas contraire, si au moins un chemin reste ouvert, le résultat retourné est `invalide`.

3 En Logique du Premier Ordre (LPO)

Le passage de la logique propositionnelle à la logique du premier ordre introduit la gestion des quantificateurs ($\forall$ et $\exists$). Cette évolution complexifie significativement la mé-

thode des connexions : il ne s'agit plus de chercher une simple égalité syntaxique stricte entre deux atomes indépendamment sur chaque chemin, mais de trouver une *substitution* globale valide (unification) qui rende ces atomes complémentaires, tout en respectant des contraintes strictes de dépendance.

3.1 Adaptations structurelles pour la LPO

Pour supporter ces nouveaux concepts sans avoir à réécrire l'intégralité de la logique de parcours, nous avons fait évoluer nos structures de base.

3.1.1 Traçabilité des variables : `var(Position, V)`

En LPO, lors de l'application d'une règle, les variables liées par des quantificateurs deviennent des variables libres dans la formule. Pour pouvoir plus tard vérifier les contraintes liées à l'ordre d'instanciation (règles δ), une simple variable Prolog anonyme ne suffit pas.

Nous avons donc encapsulé chaque variable générée dans une structure `var(NomPosition, V)`, où :

- `NomPosition` est l'index spatial unique du nœud dans l'arbre syntaxique (ex : `a1`).
- `V` est la véritable variable logique Prolog sur laquelle l'algorithme d'unification va travailler.

3.1.2 Transparence de l'arbre des chemins

L'algorithme de la section 2.2 parcourait déjà des listes de nœuds. Grâce à notre choix initial de représenter les arbres avec des nœuds n-aires (`noeud(Etiquette, ListeFils)`), nous pouvons très naturellement utiliser ces arbres pour la LPO sans rien changer.

3.2 La gestion de la Multiplicité

Contrairement à la logique propositionnelle où chaque règle ne s'applique qu'une seule fois, la règle γ (quantificateur universel $\forall$) stipule qu'une formule peut avoir besoin d'être instanciée plusieurs fois avec des termes différents pour fermer tous les chemins. Le nombre d'instanciations nécessaires est appelé **Multiplicité** (M).

3.2.1 La boucle de recherche itérative

Puisqu'il est théoriquement indécidable de connaître à l'avance la multiplicité exacte requise pour prouver une formule, nous avons mis en place une boucle d'approfondissement itératif dans le module principal (`main.pl`).

Le prédicat `verifier_boucle_lpo/3` tente de prouver la formule avec $M = 1$. En cas d'échec (et si la limite n'est pas atteinte), il relance l'intégralité de la construction de l'arbre et de la recherche avec $M = 2$, $M = 3$, etc.

```

1 % Boucle sur la multiplicite
2 verifier_boucle_lpo(Formule, M, Limite) :-
3     % Etape 1 : Arbre indexe avec multiplicite M
4     arbre_index_lpo:generer_arbre_index(Formule, M, ArbreIndexe),
5     % Etape 2 : Arbre des chemins
6     arbre_chemins_lpo:generer_arbre_chemins(ArbreIndexe,
        ArbreChemins),

```

```

7      % Etape 3 : Recherche de connexions
8      ( recherche_connexions_lpo:verifier_connexions(ArbreIndexe ,
9              ArbreChemins, valide) ->
10          format("Preuve valide avec multiplicite M = ~w.~n", [M])
11      ;
12          M1 is M + 1,
13          M1 =< Limite,
14          verifier_boucle_lpo(Formule, M1, Limite)
15      ).

```

3.2.2 Génération des copies multiples

Côté arbre indexé, le prédicat initial devient `generer_arbre_indexe/3` pour propager cette multiplicité. Lorsqu'une règle γ est rencontrée, le programme boucle pour créer M copies exactes du sous-arbre, chacune étiquetée avec un suffixe unique (`_1`, `_2`, etc.) pour garantir que les variables `var(Pos, V)` de chaque copie restent indépendantes et ne s'unifient pas accidentellement entre elles prématurément.

3.3 Unification et Graphe de Dépendance

Cependant, la recherche de connexion en LPO est plus compliquée qu'en logique propositionnelle. Trouver une paire de littéraux de polarité opposée n'est plus suffisant, il faut s'assurer que leurs arguments peuvent être unifiés globalement sur l'ensemble des chemins.

3.3.1 L'Unification intelligente

Nous avons implémenté un prédicat d'unification sur-mesure, `unifier_lpo/3`, qui gère nos structures de variables encapsulées et utilise le *occurs check* natif de Prolog pour éviter les unifications infinies ($x = f(x)$).

```

1  % Unification intelligente gerant la structure var(Position,
2      Variable)
3  unifier_lpo(var(Pos1, V), Terme, [subst(Pos1, Terme)]) :-
4      !, unify_with_occurs_check(V, Terme).
5  unifier_lpo(Terme, var(Pos2, V), [subst(Pos2, Terme)]) :-
6      !, unify_with_occurs_check(V, Terme).
7
8  % Decomposition pour les predicats complexes
9  unifier_lpo(T1, T2, SubstGlobales) :-
10      compound(T1), compound(T2), !,
11      T1 =.. [Nom | Args1],
12      T2 =.. [Nom | Args2],
13      unifier_listes_args(Args1, Args2, SubstGlobales).

```

La fonction retourne une liste de substitutions sous la forme `subst(IndexVariable, NouveauTerme)`.

3.3.2 Cohérence globale et Backtracking

En logique propositionnelle, chaque chemin était vérifié indépendamment. En LPO, nous utilisons `parcourir_chemins/2` pour accumuler toutes les substitutions nécessaires à la fermeture de **tous** les chemins dans une seule liste `ListeSubst`.

Si l'unification sur le chemin 2 nécessite que $X = a$, mais que le chemin 1 avait déjà imposé $X = b$, Prolog va automatiquement déclencher un *backtracking*. Il remontera dans l'arbre des chemins pour chercher une autre paire de feuilles sur le chemin 1, garantissant ainsi une cohérence **globale** de la preuve.

3.3.3 Validation finale : Le Graphe de dépendance

La dernière contrainte théorique forte de la méthode des connexions concerne les quantificateurs existentiels ($\exists$, règles δ). Ils introduisent des constantes de Skolem qui imposent un ordre strict d'instanciation. Pour vérifier que cet ordre est cohérent, le prédicat `verifier_connexions/3` construit un **graphe orienté de dépendance**.

Ce graphe fusionne deux types d'arêtes :

1. **Les liens structurels** (`arete(struct, P, I)`) : Ils lient l'index d'un parent à l'index de ses fils directs dans l'arbre syntaxique, représentant l'ordre sémantique de la formule.
2. **Les liens de substitution** (`arete(unif, I, V)`) : Extrait de la `ListeSubst`, ils lient l'indice d'une position présente dans un terme à la variable qui reçoit ce terme.

```
1 % Validation globale avec graphe
2 verifier_connexions(ArbreIndexe, ArbreChemins, Resultat) :-
3     % 1. Trouver une combinaison d'unification globale
4     parcourir_chemins(ArbreChemins, ListeSubst),
5     % 2. Construire le graphe de dependance
6     creer_graphe_dependance(ArbreIndexe, ListeSubst, Graphe),
7     % 3. Verifier l'absence de cycles
8     \+ detecter_cycle(Graphe),
9     !,
10    Resultat = valide.
```

Si le prédicat `detecter_cycle/1` trouve une boucle dans ce graphe (par exemple, si une variable X dépend d'une constante de Skolem c , mais que l'unification demande de remplacer X par un terme contenant c), alors la substitution est invalide. Prolog va backtracker pour trouver une autre combinaison. Si aucune combinaison acyclique n'est trouvée pour la multiplicité M , la méthode passe à $M + 1$.

Exemples et résultats

Pour valider l'exactitude de notre implémentation, nous avons testé notre programme sur une série d'exemples issus du cours et des travaux dirigés (TD). En voici une sélection démontrant le bon fonctionnement de la méthode des connexions.

Logique Propositionnelle

Exemple 1 : Transitivité de l'implication (Exemple du cours)

Formule mathématique :

$$(p \rightarrow q) \rightarrow ((q \rightarrow r) \rightarrow (p \rightarrow r))$$

Requête Prolog :

```
1 ?- trace_verif((p impl q) impl ((q impl r) impl (p impl r))).
```

```
=== Formule : ((p → q) → ((q → r) → (p → r))) ===  
  
--- Arbre syntaxique indexé ---  
((p → q) → ((q → r) → (p → r))) [a0, 0, α, _]  
├── (p → q) [a1, 1, β, α1]  
│   ├── p [a2, 0, _, β1]  
│   └── q [a3, 1, _, β2]  
└── ((q → r) → (p → r)) [a4, 0, α, α2]  
    ├── (q → r) [a5, 1, β, α1]  
    │   ├── q [a6, 0, _, β1]  
    │   └── r [a7, 1, _, β2]  
    └── (p → r) [a8, 0, α, α2]  
        ├── p [a9, 1, _, α1]  
        └── r [a10, 0, _, α2]  
  
--- Arbre des chemins ---  
{a0}  
├── {a1, a4}  
│   ├── {a2, a4}  
│   │   ├── {a5, a8, a2}  
│   │   │   ├── {a6, a8, a2}  
│   │   │   │   └── chemin final : aa9:p(1) aa10:r(0) aa6:q(0) aa2:p(0)  
│   │   │   └── {a7, a8, a2}  
│   │   │       └── chemin final : aa9:p(1) aa10:r(0) aa7:r(1) aa2:p(0)  
│   └── {a3, a4}  
│       ├── {a5, a8, a3}  
│       │   ├── {a6, a8, a3}  
│       │   │   └── chemin final : aa9:p(1) aa10:r(0) aa6:q(0) aa3:q(1)  
│       │   └── {a7, a8, a3}  
│       │       └── chemin final : aa9:p(1) aa10:r(0) aa7:r(1) aa3:q(1)  
  
--- Recherche de connexions ---  
Chemin 1 : connexion (aa9, aa2) sur 'p'.  
Chemin 2 : connexion (aa9, aa2) sur 'p'.  
Chemin 3 : connexion (aa6, aa3) sur 'q'.  
Chemin 4 : connexion (aa10, aa7) sur 'r'.  
  
--- Résultat ---  
La formule est valide.
```

Figure 1 : Résultat de l'exécution pour l'exemple 1 (Propositionnelle)

Exemple 2 : Exemple du TD

Formule mathématique :

$$((a \wedge b) \rightarrow c) \rightarrow ((a \rightarrow c) \vee (b \rightarrow c))$$

Requête Prolog :

```
1 ?- trace_verif(((a et b) impl c) impl ((a impl c) ou (b impl c))).
```

```

=== Formule : (((a ∧ b) → c) → ((a → c) ∨ (b → c))) ===

--- Arbre syntaxique indexé ---
(((a ∧ b) → c) → ((a → c) ∨ (b → c))) [a0, 0, α, _]
├── ((a ∧ b) → c) [a1, 1, β, α1]
│   ├── (a ∧ b) [a2, 0, β, β1]
│   │   ├── a [a3, 0, _, β1]
│   │   └── b [a4, 0, _, β2]
│   └── c [a5, 1, _, β2]
└── ((a → c) ∨ (b → c)) [a6, 0, α, α2]
    ├── (a → c) [a7, 0, α, α1]
    │   ├── a [a8, 1, _, α1]
    │   ├── c [a9, 0, _, α2]
    │   └── (b → c) [a10, 0, α, α2]
    │       ├── b [a11, 1, _, α1]
    │       └── c [a12, 0, _, α2]

--- Arbre des chemins ---
{a0}
├── {a1, a6}
│   ├── {a2, a6}
│   │   ├── {a3, a6}
│   │   │   ├── {a7, a10, a3}
│   │   │   │   ├── {a8, a9, a10, a3}
│   │   │   │   └── chemin final : aa11:b(1) aa12:c(0) aa8:a(1) aa9:c(0) aa3:a(0)
│   │   └── {a4, a6}
│   │       ├── {a7, a10, a4}
│   │       │   ├── {a8, a9, a10, a4}
│   │       │   └── chemin final : aa11:b(1) aa12:c(0) aa8:a(1) aa9:c(0) aa4:b(0)
│   └── {a5, a6}
│       ├── {a7, a10, a5}
│       │   ├── {a8, a9, a10, a5}
│       │   └── chemin final : aa11:b(1) aa12:c(0) aa8:a(1) aa9:c(0) aa5:c(1)

--- Recherche de connexions ---
Chemin 1 : connexion (aa8, aa3) sur 'a'.
Chemin 2 : connexion (aa11, aa4) sur 'b'.
Chemin 3 : connexion (aa12, aa5) sur 'c'.

--- Résultat ---
La formule est valide.
=== Fin ===

```

Figure 2 : Résultat de l'exécution pour l'exemple 2 (Propositionnelle)

Logique du Premier Ordre (LPO)

Exemple 3 : Exemple 1 du TD

Formule mathématique :

$$(\exists y (p(y) \wedge (\forall x (p(x) \rightarrow q(x, y)))) \rightarrow (\exists x (p(x) \wedge q(x, x))))$$

Requête Prolog :

```
1 ?- trace_verif((y ie (p(y) et (x pt (p(x) impl q(x,y)))) impl (x
    ie (p(x) et q(x,x))), lpo).
```

```
=== Formule : (∃y.(p(y) ∧ ∀x.(p(x) → q(x,y))) → ∃x.(p(x) ∧ q(x,x))) ===
>>> TENTATIVE AVEC MULTIPLICITE M = 1 <<<

--- Arbre syntaxique indexé ---
(∃y.(p(y) ∧ ∀x.(p(x) → q(x,y))) → ∃x.(p(x) ∧ q(x,x))) [a0, 0, α, _]
├── ∃y.(p(y) ∧ ∀x.(p(x) → q(x,y))) [a1, 1, δ, α1]
│   ├── (p(a2) ∧ ∀x.(p(x) → q(x,a2))) [a2, 1, α, β0]
│   │   ├── p(a2) [a3, 1, _, α1]
│   │   │   └── ∀x.(p(x) → q(x,a2)) [a4, 1, γ, α2]
│   │   │       ├── (p(a5_1) → q(a5_1,a2)) [a5_1, 1, β, γ0]
│   │   │       │   ├── p(a5_1) [a6_1, 0, _, β1]
│   │   │       │   └── q(a5_1,a2) [a7_1, 1, _, β2]
│   │   └── ∃x.(p(x) ∧ q(x,x)) [a8, 0, γ, α2]
│   │       ├── (p(a9_1) ∧ q(a9_1,a9_1)) [a9_1, 0, β, γ0]
│   │       │   ├── p(a9_1) [a10_1, 0, _, β1]
│   │       │   └── q(a9_1,a9_1) [a11_1, 0, _, β2]
└── ∃x.(p(x) ∧ q(x,x)) [a8, 0, γ, α2]

--- Arbre des chemins ---
{a0}
├── {a1, a8}
│   ├── {a2, a8}
│   │   ├── {a3, a4, a8}
│   │   │   ├── {a5_1, a3, a8}
│   │   │   │   ├── {a6_1, a3, a8}
│   │   │   │   │   ├── {a9_1, a6_1, a3}
│   │   │   │   │   │   ├── chemin final : [a10_1: p(a9_1) (0)] [a6_1: p(a5_1) (0)] [a3: p(a2) (1)]
│   │   │   │   │   │   └── chemin final : [a11_1: q(a9_1,a9_1) (0)] [a6_1: p(a5_1) (0)] [a3: p(a2) (1)]
│   │   │   │   └── {a7_1, a3, a8}
│   │   │   │       ├── {a9_1, a7_1, a3}
│   │   │   │       │   ├── chemin final : [a10_1: p(a9_1) (0)] [a7_1: q(a5_1,a2) (1)] [a3: p(a2) (1)]
│   │   │   │       │   └── chemin final : [a11_1: q(a9_1,a9_1) (0)] [a7_1: q(a5_1,a2) (1)] [a3: p(a2) (1)]
└── {a1, a8}

--- Recherche de connexions ---
Chemin 1 : connexion (a10_1, a3) entre 'p(a9_1)' et 'p(a2)'.
Substitutions : [subst(a9_1,a2)]
Chemin 2 : connexion (a6_1, a3) entre 'p(a5_1)' et 'p(a2)'.
Substitutions : [subst(a5_1,a2)]
Chemin 3 : connexion (a10_1, a3) entre 'p(a9_1)' et 'p(a2)'.
Substitutions : [subst(a9_1,a2)]
Chemin 4 : connexion (a11_1, a7_1) entre 'q(a9_1,a9_1)' et 'q(a5_1,a2)'.
Substitutions : [subst(a9_1,var(a5_1,_13598)),subst(a9_1,a2)]

--- Résultat ---
La formule est valide (Démontrée avec multiplicité M = 1).
=== Fin ===
```

Figure 3 : Résultat de l'exécution pour l'exemple 1 du TD (LPO)

Exemple 4 : Exemple 2 du TD (Dépendances et ordre)

Formule mathématique :

$$(\exists x (\forall y (p(x, y)))) \rightarrow (\forall x (\exists y (p(y, x))))$$

Requête Prolog :

```
1 ?- trace_verif((x ie (y pt (p(x,y)))) impl (x pt (y ie (p(y,x)))),
    lpo).
```

```

=== Formule : (∃x.∀y.p(x,y) → ∀x.∃y.p(y,x)) ===

>>> TENTATIVE AVEC MULTIPLICITE M = 1 <<<

--- Arbre syntaxique indexé ---
(∃x.∀y.p(x,y) → ∀x.∃y.p(y,x)) [a0, 0, α, _]
├── ∃x.∀y.p(x,y) [a1, 1, δ, α1]
│   ├── ∀y.p(a2,y) [a2, 1, γ, δ0]
│   │   ├── p(a2,a3_1) [a3_1, 1, _, γ0]
│   │   └── ∀x.∃y.p(y,x) [a4, 0, δ, α2]
│   │       ├── ∃y.p(y,a5) [a5, 0, γ, δ0]
│   │       └── p(a6_1,a5) [a6_1, 0, _, γ0]
└── ∀x.∃y.p(y,x) [a4, 0, δ, α2]

--- Arbre des chemins ---
{a0}
├── {a1, a4}
│   ├── {a2, a4}
│   │   ├── {a3_1, a4}
│   │   └── {a5, a3_1}
│   └── chemin final : [a6_1: p(a6_1,a5) (0)] [a3_1: p(a2,a3_1) (1)]

--- Recherche de connexions ---
Chemin 1 : connexion (a6_1, a3_1) entre 'p(a6_1,a5)' et 'p(a2,a3_1)'.
Substitutions : [subst(a6_1,a2),subst(a3_1,a5)]

--- Résultat ---
La formule est valide (Démontrée avec multiplicité M = 1).
=== Fin ===

```

Figure 4 : Résultat de l'exécution pour l'exemple 2 du TD (LPO) - Graphe de dépendance

Exemple 5 : Exemple 4 du TD (Nécessité de la multiplicité)

Formule mathématique :

$$\exists x (\forall y (p(y) \rightarrow p(x)))$$

Requête Prolog :

```
1 ?- trace_verif(x ie (y pt (p(y) impl p(x))), lpo).
```

```

>>> TENTATIVE AVEC MULTIPLICITE M = 2 <<<

--- Arbre syntaxique indexé ---
∃x.∀y.(p(y) → p(x)) [a0, 0, γ, _]
├── ∀y.(p(y) → p(a1_1)) [a1_1, 0, δ, γ0]
│   ├── (p(a2_1) → p(a1_1)) [a2_1, 0, α, δ0]
│   │   ├── p(a2_1) [a3_1, 1, _, α1]
│   │   └── p(a1_1) [a4_1, 0, _, α2]
│   └── ∀y.(p(y) → p(a1_2)) [a1_2, 0, δ, _]
│       ├── (p(a2_2) → p(a1_2)) [a2_2, 0, α, δ0]
│       │   ├── p(a2_2) [a3_2, 1, _, α1]
│       │   └── p(a1_2) [a4_2, 0, _, α2]
└── ∃x.∀y.(p(y) → p(x)) [a0, 0, γ, _]

--- Arbre des chemins ---
{a0}
├── {a1_1, a1_2}
│   ├── {a2_1, a1_2}
│   │   ├── {a3_1, a4_1, a1_2}
│   │   └── {a2_2, a3_1, a4_1}
│   └── chemin final : [a3_2: p(a2_2) (1)] [a4_2: p(a1_2) (0)] [a3_1: p(a2_1) (1)] [a4_1: p(a1_1) (0)]

--- Recherche de connexions ---
Chemin 1 : connexion (a3_2, a4_2) entre 'p(a2_2)' et 'p(a1_2)'.
Substitutions : [subst(a1_2,a2_2)]

--- Résultat ---
La formule est valide (Démontrée avec multiplicité M = 2).
=== Fin ===

```

Figure 5 : Résultat de l'exécution pour l'exemple 4 du TD (LPO) - Multiplicité